So in this chapter, we'll entirely focus on deploying our application on all these platforms and discuss the pros and cons of each of them. While they all are great options, it's entirely the business use case and resources which define where we deploy the application.

Deploying Flask with uWSGI and Nginx on Alibaba Cloud ECS

Deploying applications this way is often called traditional hosting where the dependencies are installed manually or through a scripted installer, which involves manually installing the application and its dependencies and securing it. In this section we'll install and run our application in production using uWSGI and Nginx on a Linux OS hosted on Alibaba Cloud Elastic Compute Service.

uWSGI is a full-fledged HTTP server and a protocol capable of running production applications. uWSGI is a popular uwsgi (protocol) server, while Nginx is a free, open source, high-performing HTTP server and a reverse proxy. In our case we'll use Nginx to reverse proxy our HTTP calls to and from the uwsgi server which we'll deploy on Ubuntu OS.

So let's get straight into the business and deploy our application, but before we do that, we have to freeze our libraries in requirements.txt using pip freeze. Run the following commands to make sure the file has the list of all the required dependencies.

```
(venv)$ pip freeze > requirements.txt
```

So here pip freeze will output all the required installed packages in requirements format. Next we need to push our codebase to a version management system like GitHub which we'll pull later on our Linux instance. For this we'll create an Ubuntu instance on Alibaba Cloud for which you can signup at `www.alibabacloud.com` or you can use your Ubuntu instance on any other cloud provider or even use a virtual one.

So before we start deploying, we also need to have a MySQL server, and since this is about deploying the Flask app, we won't be covering deploying MySQL server. However, you can deploy one on the same instance or use a managed MySQL server service and edit the DB config details in config.py.

Once you have your cloud account setup, create an Ubuntu instance preferably version 16.04 or greater. Here we have Alibaba Cloud ECS (Elastic Compute Service), and once we have our instance, we'll SSH into using keypair or a password.

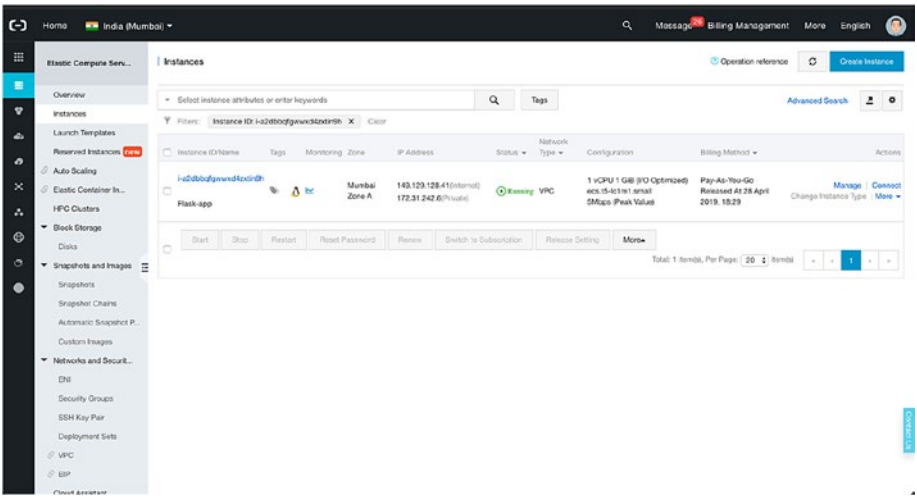

Figure 6-1. *Alibaba Cloud ECS Console*

Once you have your Ubuntu instance up and running, SSH into it and pull the codebase from your preferred version management system.

```
kunalrelan — root@iZt4nbqeosngqf68owp80aZ: ~ — ssh root@161.117.82.73 -P...
Kunals-MacBook-Pro:~ kunalrelan$ ssh root@161.117.82.73 -P
The authenticity of host '161.117.82.73 (161.117.82.73)' can't be established.
ECDSA key fingerprint is SHA256:cs6oLt9NfWzSBw+DNchFFmJVTZODbLOxyMPfy9xjKIY.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '161.117.82.73' (ECDSA) to the list of known hosts.
root@161.117.82.73's password:
Welcome to Ubuntu 16.04.6 LTS (GNU/Linux 4.4.0-146-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

Welcome to Alibaba Cloud Elastic Compute Service !

root@iZt4nbqeosngqf68owp80aZ:~#
```

Figure 6-2. *SSH into Ubuntu instance*

As you can see by default, we have logged in as a root, so before moving on, we'll create another sudo user called Flask which is a good security measure. It is a good idea to run each app under its own user account, in order to limit the damage that security vulnerabilities in the app can do.

```
$ adduser flask
```

Next it'll prompt you to set a password for the new user and enter a few details; you can just enter the password and leave other fields empty if you wish to and then run the following command to add the user to sudoers list.

```
$ usermod -aG sudo flask
```

Now once we have our new user, let's login with that user in the shell using the following command.

```
$ su - flask
```

Next we'll pull our app from our GitHub repo, so make sure you have git client installed, and if you don't, use the following command to do so.

```
$ sudo apt-get install git
```

Use the following command to clone the app repository.

```
$ sudo git clone <repo_name>
```

Next change your current directory to the app source code and install virtualenv and uwsgi since we won't have those in our reqiurements.txt.

```
$ sudo pip install virtualenv uwsgi
```

Create a virtual env like we did in the previous chapter, and install the dependencies after activating the virtual environment with the following command.

```
$ pip install -r requirements.txt
```

We'll install all the dependencies needed to set up the application from Ubuntu repositories, and we'll start with installing python-pip which is a package manager for Python and python-dev which contains the header files needed to compile Python extensions.

```
$ sudo apt-get install python-pip python-dev
```

Once we have our dependencies installed, we'll create a uWSGI configuration file which will be called flask-app.ini, so go ahead and create a file called flask-app.ini in your current directory and add the following lines to it.

```
[uwsgi]
module = run:application

master = true
processes = 5

socket = flask-app.sock
chmod-socket = 660
vacuum = true

die-on-term = true
```

This file starts with [uwsgi] header so that WSGI knows to apply the settings. We also specify the module and the callable which is run.py in our case minus the extension and the callable which is application.

Then we instruct uwsgi to start the process as a master and spawn five worker processes to handle the requests.

Next we'll supply the Unix socket file for Nginx to follow the uWSGI requests for our application. Let's also change the permissions on the socket. We'll be giving the Nginx group ownership of the uWSGI process later on, so we need to make sure the group owner of the socket can read information from it and write to it. We will also clean up the socket when the process stops by adding the vacuum option.

The last thing we'll do is set the die-on-term option. This can help ensure that the init system and uWSGI have the same assumptions about what each process signal means.

Next, we'll create a systemd service unit file which will allow Ubuntu's init system to start our application automatically whenever the server boots.

```
This file will be called flask-app.service and will be placed
in /etc/systemd/system
Directory.
$ sudo nano /etc/systemd/system/flask-app.service
```

And paste the following lines in the file.

```
#Metadata and dependencies section
[Unit]
Description=Flask App service
After=network.target
#Define users and app working directory
[Service]
User=flask
Group=www-data
```

```
WorkingDirectory=/home/flask/flask-api-app/src
Environment="WORK_ENV=PROD"
ExecStart=/home/flask/flask-api-app/src/venv/bin/uwsgi --ini
flask-app.ini
#Link the service to start on multi-user system up
[Install]
WantedBy=multi-user.target
```

After this run the following command to enable and start our new service.

```
$ sudo systemctl start flask-app
$ sudo systemctl enable flask-app
```

Our uWSGI server should now be up and running waiting for requests in the socket file we made earlier. We'll now install and configure Nginx to pass and process the requests using the uwsgi protocol.

```
$ sudo apt-get install nginx
```

Now we should have an Nginx server up and running, and we'll begin by creating a new server block config file in /etc/nginx/sites-available , and we'll call it flask-app.

```
$ sudo nano /etc/nginx/sites-available/flask-app
```

We'll open a server block and instruct it to listen on port 80 and define the server name which should be the domain name of your service. Going ahead we'll define a location block inside the server block to define the base location and import uwsgi_params headers inside that specifies some general uWSGI parameters that need to be set. We'll then pass the requests to the socket we defined using the uwsgi_pass directive.

```
server {
    listen 80;
```

```
    server_name flaskapp;

    location / {
        include uwsgi_params;
        uwsgi_pass unix:/home/flask/flask-api-app/src/flask-
        app.sock;
    }
}
```

The preceding lines should configure our server block to listen to server requests on the socket. Once we have that all ready, next we'll create a symlink to sites-enabled directory.

```
$ sudo ln -s /etc/nginx/sites-available/flask-app /etc/nginx/
sites-enabled
```

With that in place, we can now test our changes for syntax errors using the following command.

```
$ sudo nginx -t
```

If there were no syntax errors, you should see this printed in your terminal.

```
$ nginx: the configuration file /etc/nginx/nginx.conf syntax is
ok
$ nginx: configuration file /etc/nginx/nginx.conf test is
successful
```

And now when you access your domain, it should be working. In case you don't have a domain and want to access the application, you should either edit the default server block in sites-enabled rather than creating a new one, or delete the default app and you can just access the application with the IP address.